Database Management System

Project Report

MediCare Hospital Management System

Subject Code: 25CSH-204

University Institute of Engineering (UIE)

Department of Computer Science & Engineering

Submitted by:

Team 3 | Program: BE-CSE

Name: Ishika Pandey
UID: 25BCS11688

Name: Bhavya Baid
UID: 25BCS10268

Name: Arjita Shrivastava
UID: 25BCS12986

Submitted to:

Name: Mr. Gaurav Kumar Sen

Subject: Database Management System

Subject Code: 25CSH-204

Acknowledgement

We would like to express our sincere gratitude to **Chandigarh University** and the **University Institute of Engineering** for giving us the opportunity to work on this project as a part of the Database Management System course. Working on a complete database from the first design to the last query helped us understand many topics that were earlier only theory for us.

We are especially thankful to our subject teacher, **Mr. Gaurav Kumar Sen**, for his guidance, patience and encouragement. His lectures on the relational model, normalization and SQL gave us the base on which this project stands, and his suggestions made us think more carefully about the design.

We also thank each other. The three of us divided the work, tested each other's queries and sorted out the mistakes together, which made the project smoother and more enjoyable. Finally, we thank our parents and friends for their constant support during the semester.

Team 3

Name: Ishika Pandey | UID: 25BCS11688

Name: Bhavya Baid | UID: 25BCS10268

Name: Arjita Shrivastava | UID: 25BCS12986

BE-CSE | Semester 3 | Chandigarh University

Index

S.No	Chapter	Page No.
1	Acknowledgement	2
2	Abstract	4
3	Chapter 1: Introduction	5
4	Chapter 2: Problem Statement and Objectives	6
5	Chapter 3: Scope of the Project	6
6	Chapter 4: Requirement Analysis	8
7	Chapter 5: ER Diagram	9
8	Chapter 6: Relational Schema / Database Design	10
9	Chapter 7: Normalization (1NF, 2NF, 3NF)	11
10	Chapter 8: SQL Implementation	13
	8.1 Data Definition Language (DDL)	13
	8.2 Data Manipulation Language (DML)	15
	8.3 Data Query Language (DQL)	18
	8.4 Transaction Control Language (TCL)	19
	8.5 Data Control Language (DCL)	20
11	Chapter 9: SQL Queries with Explanation and Output	21
	9.1 Basic retrieval	21
	9.2 Aggregate functions	22
	9.3 Joins	23
	9.4 Subqueries	25
	9.5 Views and CASE	26
12	Chapter 10: Constraints, Joins, Subqueries and Aggregate Functions	27
13	Chapter 11: Screenshots of Execution	28
14	Chapter 12: Conclusion	30
15	Chapter 13: Future Scope	30
16	Chapter 14: References	30

Abstract

Item	Details
Project title	MediCare Hospital Management System
Domain	Healthcare (hospital database)
Database / Tool	MySQL 8.0 and MySQL Workbench
Subject	Database Management System (25CSH-204)
Team	Team 3: Ishika Pandey (25BCS11688), Bhavya Baid (25BCS10268), Arjita Shrivastava (25BCS12986)

Every hospital runs on information. When a patient walks in, someone has to note down who the patient is, which doctor will see them, what medicines are prescribed, whether a bed is needed and finally how much has to be paid. In many hospitals this still happens in registers or separate spreadsheets, and the result is familiar to anyone who has stood in a hospital queue: repeated forms, lost files and bills that do not match the treatment.

This project, **MediCare Hospital Management System**, shows how a relational database can keep all this information in one organised place. We designed a database named `medicare_db` for a fictional hospital and implemented it in MySQL 8.0. It has nine tables: department, doctor, patient, appointment, room, admission, medicine, prescription and bill, connected through primary and foreign keys. Constraints such as NOT NULL, UNIQUE, CHECK and DEFAULT keep the stored data valid.

The report follows the complete design process: requirements, ER diagram, relational schema and normalization up to 3NF, followed by implementation with DDL, DML, DQL, TCL and DCL commands. Twenty queries covering filtering, aggregate functions, joins, subqueries, views and CASE show how the data answers everyday hospital questions. Every query was executed in MySQL Workbench, and its output and screenshots are included.

Keywords: DBMS, MySQL, Hospital Management, ER Diagram, Normalization, SQL, Healthcare.

Chapter 1: Introduction

1.1 Background

Healthcare is one of the few fields where wrong or missing data can directly affect a person's health. A doctor must know a patient's earlier visits before prescribing a medicine, the pharmacy must know what is in stock, and the accounts section must know what has been billed. All of these people work on the same facts about the same patients, but each looks at those facts from a different side.

Paper files create problems here. The same patient's address may be written differently in three registers, an old record can take minutes to find, and a simple question such as "which medicines are about to run out?" cannot be answered without counting by hand. A database management system solves this by storing every fact only once and letting each department read it the way it needs [1].

1.2 DBMS and the Relational Model

A Database Management System (DBMS) is software that stores, organises and protects data and lets many users retrieve or change it safely. In the relational model used by MySQL, Oracle and PostgreSQL, data is kept in tables of rows and columns. A row represents one real-world thing, such as a patient, and a column represents one property of it, such as the city. Tables are linked using keys and are handled with SQL (Structured Query Language) [2]. The main benefits are less redundancy, data integrity through constraints, fast retrieval with indexes, security through user privileges and safe sharing through transactions.

1.3 About the Project

MediCare is the name we gave to a fictional hospital with six departments: Cardiology, Neurology, Orthopedics, Pediatrics, General Medicine and Dermatology. The database records its doctors, registered patients, appointments, prescribed medicines, room admissions and bills. We chose this domain because it contains every situation a good database project needs: one-to-many relationships (a department has many doctors), a many-to-many relationship (an appointment can have many medicines), optional values (a patient not yet discharged) and many reporting needs. All names, numbers and amounts in this report are made up for demonstration.

1.4 Tools and Technologies Used

Purpose	Tool / Technology
Database server	MySQL 8.0 Community Server
Client	MySQL Workbench 8.0
Language	SQL (DDL, DML, DQL, TCL and DCL)
ER diagram	Graphviz
Version control	Git and GitHub

1.5 Literature Review

Hospital information systems have moved from paper registers to file-based programs and finally to relational databases. File-based systems suffered from duplicate data and inconsistency, which led to the relational model proposed by E. F. Codd and later to SQL [3]. Modern hospital software still rests on the same ideas: normalized tables, keys and constraints, and controlled access. This project applies these basic principles on a small scale, and the full source is available in our GitHub repository.

Chapter 2: Problem Statement and Objectives

2.1 Problem Statement

Problem Statement

To design and implement a relational database for a hospital that stores the details of departments, doctors, patients, appointments, room admissions, medicines, prescriptions and bills, keeps this data consistent and free from duplication, and answers common administrative and medical questions through SQL queries.

2.2 Problems in the Manual System

Problem	What it causes	How the database helps
Data redundancy	Same details written again and again	Patient stored once, referred to by patient_id
Slow searching	Old records take minutes to find	Primary keys and indexes give quick lookup
Wrong billing	Amounts do not match the treatment	Bills are linked to patients
No stock tracking	Medicines run out without warning	Stock checked with a simple query
Poor security	Anyone can open any register	Users get only the privileges they need
Hard reporting	Figures counted by hand	Aggregate queries give reports in seconds

2.3 Objectives

1. To study the working of a hospital and identify its entities and relationships.
2. To draw an ER diagram and convert it into a relational schema with correct keys.
3. To normalise the design up to the Third Normal Form.
4. To create the database with DDL commands and protect it with constraints.
5. To fill and modify the data with DML and control transactions with TCL.
6. To write twenty queries using joins, subqueries, aggregate functions, views and CASE.
7. To create users with limited privileges using DCL commands.

Chapter 3: Scope of the Project

3.1 What the Project Covers

- Registration and storage of patient details.
- Doctor and department information with salary and consultation fee.
- Booking, completing and cancelling appointments.
- Prescribing medicines and keeping the pharmacy stock.
- Room allotment, admission and discharge of in-patients.
- Billing with payment mode and payment status, and management reports.

3.2 Limitations

- The project is a database design; there is no graphical front end and all work is done through SQL.
- Time slots for appointments, lab tests and insurance claims are not modelled.

- The data is sample data. A real hospital would need far more records and legal care for medical data.

3.3 Users of the System

User	Main work	Tables mostly used
Receptionist	Registers patients, books appointments	patient, appointment
Doctor	Views history, writes prescriptions	appointment, prescription
Pharmacist	Checks and updates medicine stock	medicine, prescription
Billing staff	Creates bills, records payments	bill, admission, room
Administrator	Manages doctors, departments, reports	all tables

Chapter 4: Requirement Analysis

4.1 Functional Requirements

ID	Requirement
FR-1	Store departments with a unique name and doctors linked to one department.
FR-2	Register patients with gender, date of birth, city and blood group.
FR-3	Record appointments between a patient and a doctor with a status.
FR-4	Record medicines prescribed in an appointment with dosage and duration.
FR-5	Maintain medicine price and stock quantity.
FR-6	Allot rooms to patients and record admission and discharge dates.
FR-7	Generate bills with payment mode and status.
FR-8	Provide reports such as revenue, busiest department and low stock.
FR-9	Restrict access according to the role of the user.

4.2 Non-Functional Requirements

- **Integrity:** wrong values such as a negative salary or an unknown patient must be rejected.
- **Performance:** frequently searched columns (city, appointment date) are indexed.
- **Security:** access is controlled through user accounts and privileges.
- **Portability:** the script must run on any machine with MySQL 8.0.

4.3 Hardware and Software Requirements

Item	Minimum requirement
Processor and RAM	Intel Core i3 or equivalent, 4 GB RAM
Disk space	2 GB free
Operating system	Windows 10 / 11, Linux or macOS
Software	MySQL Server 8.0 and MySQL Workbench 8.0

Chapter 5: ER Diagram

The Entity-Relationship diagram is the blueprint of the database. Rectangles are entities with their attributes, diamonds are relationships and the letters on the lines show cardinality. Underlined attributes are primary keys and italic ones are foreign keys. The many-to-many relationship **prescribes** has its own attributes (dosage and duration_days), so it becomes the separate table `prescription`.

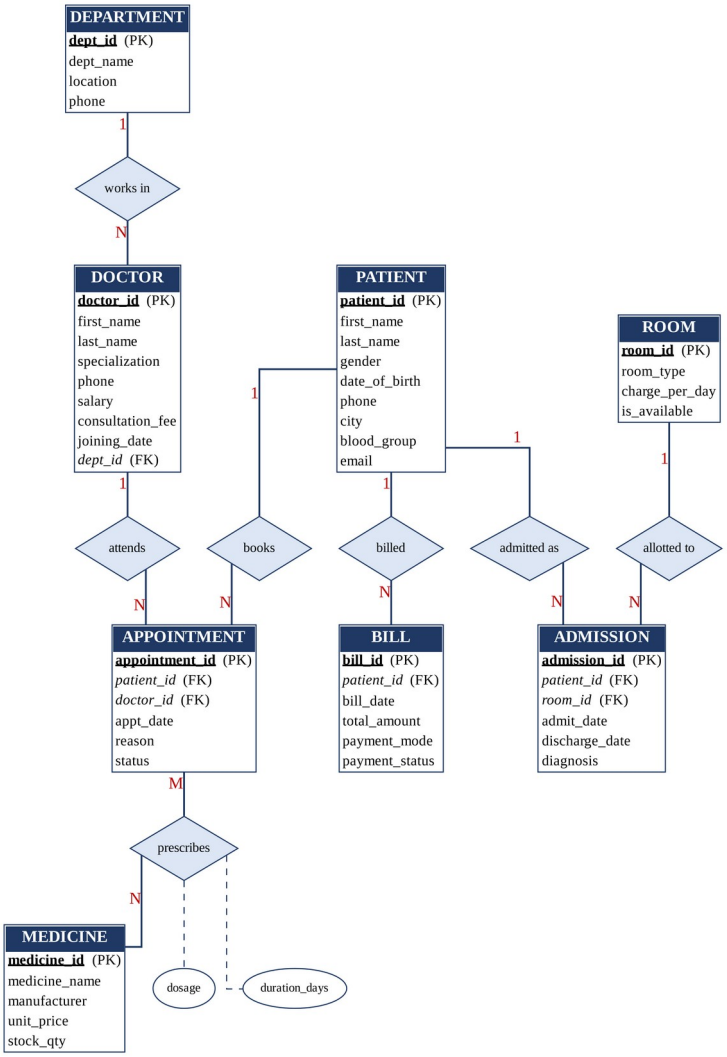


Figure 5.1: ER diagram of the MediCare Hospital Management System

5.1 Relationships and Cardinality

Relationship	Entities	Cardinality	Meaning
works in	Department – Doctor	1 : N	One department has many doctors
attends	Doctor – Appointment	1 : N	A doctor attends many appointments
books	Patient – Appointment	1 : N	A patient books many appointments
prescribes	Appointment – Medicine	M : N	Medicines advised in an appointment
admitted as	Patient – Admission	1 : N	A patient can be admitted many times
allotted to	Room – Admission	1 : N	A room is used for many admissions
billed	Patient – Bill	1 : N	A patient receives many bills

Chapter 6: Relational Schema / Database Design

6.1 Mapping the ER Diagram to Tables

Every entity became a table with its key attribute as primary key. For each one-to-many relationship, the primary key of the "one" side was added to the "many" side as a foreign key. The many-to-many relationship was converted into the separate table `prescription`, which holds the keys of both sides and the attributes of the relationship.

6.2 Relational Schema and Data Dictionary

Table	Columns	Keys and constraints
department	dept_id (PK), dept_name, location, phone	dept_name NOT NULL UNIQUE
doctor	doctor_id (PK), first_name, last_name, specialization, phone, salary, consultation_fee, joining_date, dept_id (FK)	phone UNIQUE; salary > 0; fee >= 0; dept_id references department
patient	patient_id (PK), first_name, last_name, gender, date_of_birth, phone, city, blood_group, email	gender IN (M, F, O); phone UNIQUE; index on city
appointment	appointment_id (PK), patient_id (FK), doctor_id (FK), appt_date, reason, status	status has 3 valid values, DEFAULT Scheduled; index on appt_date
room	room_id (PK), room_type, charge_per_day, is_available	room_type has 4 valid values; charge > 0; DEFAULT Y
admission	admission_id (PK), patient_id (FK), room_id (FK), admit_date, discharge_date, diagnosis	discharge_date is NULL or >= admit_date
medicine	medicine_id (PK), medicine_name, manufacturer, unit_price, stock_qty	name UNIQUE; price >= 0; stock >= 0
prescription	prescription_id (PK), appointment_id (FK), medicine_id (FK), dosage, duration_days	duration > 0; ON DELETE CASCADE on appointment
bill	bill_id (PK), patient_id (FK), bill_date, total_amount, payment_mode, payment_status	amount >= 0; status has 3 valid values, DEFAULT Pending

Chapter 7: Normalization (1NF, 2NF, 3NF)

Normalization organises data so that every fact is stored at one place only. It removes the insertion, deletion and update anomalies that appear when unrelated facts are mixed in one table [3]. We start with a register-style table and normalise it step by step.

7.1 Unnormalized Table (UNF)

Each row is one visit. The medicines column holds several values, and doctor, department and patient details repeat for every visit.

visit_id	patient	phone	doctor	dept_name	location	medicines (dosage)
1	Amit Kapoor	9812300001	Rajesh Kumar	Cardiology	Block A	Atorvastatin (Once daily), Aspirin (Once daily)
4	Rohan Mehta	9812300003	Meera Nair	Pediatrics	Block A	Paracetamol (Thrice daily), Amoxicillin (Twice daily)

7.2 First Normal Form (1NF)

1NF needs atomic values and no repeating groups. Each row now holds one medicine, and the key becomes (visit_id, medicine).

visit_id	patient	phone	doctor	dept_name	location	medicine	dosage
1	Amit Kapoor	9812300001	Rajesh Kumar	Cardiology	Block A	Atorvastatin	Once daily
1	Amit Kapoor	9812300001	Rajesh Kumar	Cardiology	Block A	Aspirin	Once daily
4	Rohan Mehta	9812300003	Meera Nair	Pediatrics	Block A	Paracetamol	Thrice daily

7.3 Second Normal Form (2NF)

2NF needs 1NF and no partial dependency on part of a composite key. Patient, doctor and department details depend only on visit_id, so the table is split in two.

VISIT: visit_id	patient	phone	doctor	dept_name	location
1	Amit Kapoor	9812300001	Rajesh Kumar	Cardiology	Block A
4	Rohan Mehta	9812300003	Meera Nair	Pediatrics	Block A

VISIT_MEDICINE: visit_id	medicine	dosage
1	Atorvastatin	Once daily
1	Aspirin	Once daily
4	Paracetamol	Thrice daily

7.4 Third Normal Form (3NF)

3NF needs 2NF and no transitive dependency. In VISIT we have visit → doctor → dept_name → location and visit → patient → phone. These chains are broken into separate tables, and names are replaced by ids. Medicine details also get their own table so the price is stored once. This gives the final tables of the project.

Final table	Columns after normalization	Dependency removed
patient	patient_id, name, phone, city ...	patient → phone
department	dept_id, dept_name, location	doctor → dept_name → location
doctor	doctor_id, name, specialization, dept_id	visit → doctor → department
appointment	appointment_id, patient_id, doctor_id, appt_date, status	Repeated visit details
medicine	medicine_id, medicine_name, unit_price, stock_qty	medicine → price
prescription	prescription_id, appointment_id, medicine_id, dosage	Partial dependency

Every determinant in the nine final tables is a candidate key, so the design also satisfies BCNF.

Chapter 8: SQL Implementation

All statements below were written in one query tab of MySQL Workbench and executed in the same order. The complete script is also available in the `sql` folder of our GitHub repository.

8.1 Data Definition Language (DDL)

DDL commands define the structure of the database: CREATE, ALTER, DROP and TRUNCATE. Parent tables are created before child tables because a foreign key can only point to an existing table. Each CREATE TABLE returns "Query OK, 0 rows affected".

Creating the database

Every object of the project lives inside one database. IF NOT EXISTS lets the script be run again without an error.

```
CREATE DATABASE IF NOT EXISTS medicare_db;
USE medicare_db;
```

Output:

```
Query OK, 1 row affected (0.01 sec)

Database changed
```

Table 1: department

Stores the clinical departments of the hospital. The department name is UNIQUE so the same department cannot be entered twice.

```
CREATE TABLE department (
    dept_id    INT PRIMARY KEY,
    dept_name  VARCHAR(50) NOT NULL UNIQUE,
    location   VARCHAR(50) NOT NULL,
    phone      VARCHAR(15)
);
```

Table 2: doctor

Every doctor belongs to exactly one department, so dept_id is a foreign key. The CHECK constraints stop negative salary or fee values from entering the table.

```
CREATE TABLE doctor (
    doctor_id         INT PRIMARY KEY,
    first_name        VARCHAR(30) NOT NULL,
    last_name         VARCHAR(30) NOT NULL,
    specialization    VARCHAR(50) NOT NULL,
    phone             VARCHAR(15) UNIQUE,
    salary            DECIMAL(10,2) NOT NULL,
    consultation_fee  DECIMAL(8,2) NOT NULL,
    joining_date      DATE NOT NULL,
    dept_id           INT NOT NULL,
    CONSTRAINT chk_doctor_salary CHECK (salary > 0),
    CONSTRAINT chk_doctor_fee    CHECK (consultation_fee >= 0),
    CONSTRAINT fk_doctor_dept FOREIGN KEY (dept_id)
        REFERENCES department(dept_id)
);
```

Table 3: patient

Holds the personal details of every registered patient. Gender is restricted to M, F or O using a CHECK constraint, and the phone number is unique for each patient.

```
CREATE TABLE patient (
    patient_id     INT PRIMARY KEY,
    first_name     VARCHAR(30) NOT NULL,
    last_name      VARCHAR(30) NOT NULL,
    gender         CHAR(1) NOT NULL,
    date_of_birth  DATE NOT NULL,
    phone          VARCHAR(15) UNIQUE,
    city           VARCHAR(40),
    blood_group    VARCHAR(3),
    CONSTRAINT chk_patient_gender CHECK (gender IN ('M','F','O'))
);
```

Table 4: appointment

Links a patient with a doctor on a particular date. The status column has a DEFAULT value of Scheduled and can only take three valid values.

```
CREATE TABLE appointment (
    appointment_id  INT PRIMARY KEY,
    patient_id      INT NOT NULL,
    doctor_id       INT NOT NULL,
    appt_date       DATE NOT NULL,
    reason          VARCHAR(100),
    status          VARCHAR(10) NOT NULL DEFAULT 'Scheduled',
    CONSTRAINT chk_appt_status CHECK
        (status IN ('Scheduled','Completed','Cancelled')),
    CONSTRAINT fk_appt_patient FOREIGN KEY (patient_id)
        REFERENCES patient(patient_id),
    CONSTRAINT fk_appt_doctor FOREIGN KEY (doctor_id)
        REFERENCES doctor(doctor_id)
);
```

Table 5: room

Contains the wards and rooms available for admission, with the daily charge for each room type.

```
CREATE TABLE room (
    room_id         INT PRIMARY KEY,
    room_type       VARCHAR(20) NOT NULL,
    charge_per_day  DECIMAL(8,2) NOT NULL,
    is_available    CHAR(1) NOT NULL DEFAULT 'Y',
    CONSTRAINT chk_room_type CHECK
        (room_type IN ('General','Semi-Private','Private','ICU')),
    CONSTRAINT chk_room_charge CHECK (charge_per_day > 0)
);
```

Table 6: admission

Records every stay of a patient in a room. A NULL discharge_date means that the patient is still admitted. The date CHECK makes sure a patient cannot be discharged before being admitted.

```
CREATE TABLE admission (
    admission_id    INT PRIMARY KEY,
    patient_id      INT NOT NULL,
    room_id         INT NOT NULL,
    admit_date      DATE NOT NULL,
    discharge_date  DATE,
    diagnosis       VARCHAR(100),
    CONSTRAINT chk_admission_dates CHECK
        (discharge_date IS NULL OR discharge_date >= admit_date),
    CONSTRAINT fk_adm_patient FOREIGN KEY (patient_id)
        REFERENCES patient(patient_id),
    CONSTRAINT fk_adm_room FOREIGN KEY (room_id)
        REFERENCES room(room_id)
);
```

Table 7: medicine

The pharmacy stock. Price and stock quantity are validated with CHECK constraints.

```
CREATE TABLE medicine (
    medicine_id    INT PRIMARY KEY,
    medicine_name  VARCHAR(50) NOT NULL UNIQUE,
    manufacturer   VARCHAR(50),
    unit_price     DECIMAL(8,2) NOT NULL,
    stock_qty      INT NOT NULL DEFAULT 0,
    CONSTRAINT chk_med_price CHECK (unit_price >= 0),
    CONSTRAINT chk_med_stock CHECK (stock_qty >= 0)
);
```

Table 8: prescription

This is the bridge table that resolves the many-to-many relationship between appointment and medicine. ON DELETE CASCADE removes the prescription lines automatically if their appointment is deleted.

```
CREATE TABLE prescription (
    prescription_id  INT PRIMARY KEY,
    appointment_id   INT NOT NULL,
    medicine_id      INT NOT NULL,
    dosage           VARCHAR(30) NOT NULL,
    duration_days    INT NOT NULL,
    CONSTRAINT chk_presc_days CHECK (duration_days > 0),
    CONSTRAINT fk_presc_appt FOREIGN KEY (appointment_id)
        REFERENCES appointment(appointment_id) ON DELETE CASCADE,
    CONSTRAINT fk_presc_med FOREIGN KEY (medicine_id)
        REFERENCES medicine(medicine_id)
);
```

Table 9: bill

Keeps the billing and payment details of each patient. Payment status can only be Paid, Pending or Partial.

```
CREATE TABLE bill (
    bill_id         INT PRIMARY KEY,
    patient_id      INT NOT NULL,
    bill_date       DATE NOT NULL,
    total_amount    DECIMAL(10,2) NOT NULL,
    payment_mode    VARCHAR(15),
    payment_status  VARCHAR(10) NOT NULL DEFAULT 'Pending',
    CONSTRAINT chk_bill_amount CHECK (total_amount >= 0),
    CONSTRAINT chk_bill_status CHECK
        (payment_status IN ('Paid','Pending','Partial')),
    CONSTRAINT fk_bill_patient FOREIGN KEY (patient_id)
        REFERENCES patient(patient_id)
);
```

ALTER, CREATE INDEX and DROP

ALTER TABLE adds a new column to an existing table. Indexes speed up searches on columns that are used often in WHERE clauses. A temporary table is created, emptied with TRUNCATE and finally removed with DROP to show the remaining DDL commands.

```
ALTER TABLE patient ADD COLUMN email VARCHAR(60);

CREATE INDEX idx_patient_city ON patient(city);
CREATE INDEX idx_appt_date ON appointment(appt_date);

CREATE TABLE temp_feedback (
    feedback_id  INT PRIMARY KEY,
    comments     VARCHAR(100)
);
TRUNCATE TABLE temp_feedback;
DROP TABLE temp_feedback;
```

Output:

```
Query OK, 0 rows affected (0.06 sec)
Records: 0  Duplicates: 0  Warnings: 0

Query OK, 0 rows affected (0.03 sec)
Records: 0  Duplicates: 0  Warnings: 0

Query OK, 0 rows affected (0.03 sec)
Records: 0  Duplicates: 0  Warnings: 0

Query OK, 0 rows affected (0.04 sec)

Query OK, 0 rows affected (0.05 sec)

Query OK, 0 rows affected (0.03 sec)
```

8.2 Data Manipulation Language (DML)

DML commands add, change and remove data: INSERT, UPDATE and DELETE. Parent tables are filled first so every foreign key finds its row. Each INSERT returns "Query OK" with the number of rows added.

INSERT: department

Six departments are inserted with a single multi-row INSERT statement.

```
INSERT INTO department VALUES
(1, 'Cardiology',       'Block A', '0172-4001001'),
(2, 'Neurology',        'Block B', '0172-4001002'),
(3, 'Orthopedics',      'Block C', '0172-4001003'),
(4, 'Pediatrics',       'Block A', '0172-4001004'),
(5, 'General Medicine', 'Block D', '0172-4001005'),
(6, 'Dermatology',      'Block B', '0172-4001006');
```

INSERT: doctor

Eight doctors are added. The last value of every row is the dept_id of the department the doctor works in.

```
INSERT INTO doctor VALUES
(101,'Rajesh','Kumar','Cardiologist','9876500101',150000,800,'2015-06-12',1),
(102,'Anjali','Sharma','Neurologist','9876500102',140000,900,'2017-03-20',2),
(103,'Vikram','Singh','Orthopedic Surgeon','9876500103',135000,700,'2016-08-01',3),
(104,'Meera','Nair','Pediatrician','9876500104',95000,500,'2019-01-15',4),
(105,'Sandeep','Verma','General Physician','9876500105',85000,400,'2020-07-10',5),
(106,'Priya','Malhotra','Dermatologist','9876500106',90000,600,'2021-02-25',6),
(107,'Arvind','Chauhan','Cardiologist','9876500107',120000,750,'2018-11-05',1),
(108,'Neha','Gupta','General Physician','9876500108',80000,400,'2022-04-18',5);
```

INSERT: patient

Twelve patients from different cities are registered. The email column added earlier is not listed, so it stays NULL for now.

```
INSERT INTO patient
(patient_id, first_name, last_name, gender, date_of_birth, phone, city, blood_group)
VALUES
(1, 'Amit',   'Kapoor',    'M', '1985-04-12', '9812300001', 'Chandigarh', 'B+'),
(2, 'Sneha',  'Reddy',     'F', '1992-09-30', '9812300002', 'Mohali',     'O+'),
(3, 'Rohan',  'Mehta',     'M', '2015-02-18', '9812300003', 'Panchkula',  'A+'),
(4, 'Pooja',  'Yadav',     'F', '1978-12-05', '9812300004', 'Rewari',     'B-'),
(5, 'Karan',  'Malhotra',  'M', '1969-07-21', '9812300005', 'Ambala',     'AB+'),
(6, 'Simran', 'Kaur',      'F', '2000-01-14', '9812300006', 'Chandigarh', 'O-'),
(7, 'Deepak', 'Joshi',     'M', '1958-03-09', '9812300007', 'Mohali',     'A-'),
(8, 'Kavita', 'Bansal',    'F', '1988-11-27', '9812300008', 'Panchkula',  'B+'),
(9, 'Harsh',  'Vardhan',   'M', '2010-06-02', '9812300009', 'Rewari',     'O+'),
(10,'Anita',  'Choudhary', 'F', '1975-05-16', '9812300010', 'Ambala',     'A+'),
(11,'Manish', 'Tiwari',    'M', '1995-08-08', '9812300011', 'Chandigarh', 'B+'),
(12,'Ritu',   'Saxena',    'F', '1983-10-23', '9812300012', 'Mohali',     'AB-');
```

INSERT: room

Six rooms of four types are inserted. The is_available column is not mentioned, so MySQL fills in the default value Y.

```
INSERT INTO room (room_id, room_type, charge_per_day) VALUES
(1, 'General',      1500),
(2, 'General',      1500),
(3, 'Semi-Private', 2500),
(4, 'Private',      4000),
(5, 'Private',      4000),
(6, 'ICU',          8000);
```

INSERT: appointment

Fifteen appointments are loaded: twelve completed, one cancelled and two upcoming ones.

```
INSERT INTO appointment VALUES
(1,  1, 101, '2026-09-05', 'Chest pain',            'Completed'),
(2,  5, 107, '2026-09-10', 'Severe chest pain',     'Completed'),
(3,  7, 102, '2026-09-20', 'Sudden weakness',       'Completed'),
(4,  3, 104, '2026-09-22', 'Fever and cough',       'Completed'),
(5,  6, 106, '2026-09-25', 'Skin allergy',          'Completed'),
(6,  4, 103, '2026-10-02', 'Leg injury',            'Completed'),
(7,  11,105, '2026-10-05', 'Seasonal flu',          'Completed'),
(8,  8, 102, '2026-10-12', 'Recurring migraine',    'Completed'),
(9,  2, 108, '2026-10-18', 'High fever',            'Completed'),
(10, 9, 104, '2026-10-25', 'Vaccination check',     'Completed'),
(11, 12,106, '2026-10-30', 'Skin rash',             'Cancelled'),
(12, 10,105, '2026-11-01', 'Breathing difficulty',  'Completed'),
(13, 12,101, '2026-11-04', 'Routine heart checkup', 'Completed'),
(14, 1, 107, '2026-11-10', 'Follow-up visit',       'Scheduled'),
(15, 6, 106, '2026-11-12', 'Skin allergy follow-up','Scheduled');
```

INSERT: admission

Five admissions are recorded. Patient 10 is still in the hospital, so her discharge date is NULL.

```
INSERT INTO admission VALUES
(1, 5,  6, '2026-09-10', '2026-09-15', 'Acute myocardial infarction'),
(2, 7,  4, '2026-09-20', '2026-09-26', 'Ischemic stroke'),
(3, 4,  3, '2026-10-02', '2026-10-05', 'Fractured tibia'),
(4, 2,  1, '2026-10-18', '2026-10-20', 'Dengue fever'),
(5, 10, 5, '2026-11-01', NULL,         'Pneumonia');
```

INSERT: medicine

Nine commonly used medicines are stocked in the pharmacy table.

```
INSERT INTO medicine VALUES
(1, 'Paracetamol',   'Cipla',       5.00,  500),
(2, 'Amoxicillin',   'Sun Pharma', 12.00,  300),
(3, 'Cetirizine',    'Zydus',       4.00,  400),
(4, 'Atorvastatin',  'Lupin',      18.50,  250),
(5, 'Aspirin',       'Cipla',       3.00,  600),
(6, 'Ibuprofen',     'Dr. Reddys',  6.00,   80),
(7, 'Pantoprazole',  'Sun Pharma',  9.50,  350),
(8, 'Azithromycin',  'Zydus',      25.00,   60),
(9, 'Amlodipine',    'Lupin',       8.00,  220);
```

INSERT: prescription

Eighteen prescription lines connect appointments with the medicines that were advised.

```
INSERT INTO prescription VALUES
(1,  1, 4, 'Once daily',   30),
(2,  1, 5, 'Once daily',   30),
(3,  2, 4, 'Once daily',   30),
(4,  2, 9, 'Once daily',   30),
(5,  2, 5, 'Once daily',   30),
(6,  3, 5, 'Once daily',   30),
(7,  4, 1, 'Thrice daily',  5),
(8,  4, 2, 'Twice daily',   5),
(9,  5, 3, 'Once daily',    7),
(10, 6, 6, 'Twice daily',   7),
(11, 6, 7, 'Once daily',    7),
(12, 7, 1, 'Thrice daily',  3),
(13, 7, 3, 'Once daily',    5),
(14, 8, 6, 'Twice daily',   5),
(15, 9, 1, 'Thrice daily',  5),
(16, 12,8, 'Once daily',    3),
(17, 12,7, 'Once daily',    5),
(18, 13,9, 'Once daily',   30);
```

INSERT: bill

Twelve bills are created. Bill 12 belongs to the patient who is currently admitted, so it is Pending and has no payment mode yet.

```
INSERT INTO bill VALUES
(1,  5, '2026-09-15', 48500.00, 'Card',      'Paid'),
(2,  7, '2026-09-26', 36200.00, 'Insurance', 'Paid'),
(3,  4, '2026-10-05', 17850.00, 'UPI',       'Paid'),
(4,  2, '2026-10-20',  6450.00, 'Cash',      'Paid'),
(5,  1, '2026-09-05',  1250.00, 'UPI',       'Paid'),
(6,  3, '2026-09-22',   650.00, 'Cash',      'Paid'),
(7,  6, '2026-09-25',   750.00, 'UPI',       'Paid'),
(8, 11,'2026-10-05',   550.00, 'Cash',      'Paid'),
(9,  8, '2026-10-12',  1100.00, 'Card',      'Partial'),
(10, 9, '2026-10-25',   500.00, 'Cash',      'Paid'),
(11, 12,'2026-11-04',   950.00, 'UPI',       'Paid'),
(12, 10,'2026-11-01',  2400.00, NULL,        'Pending');
```

UPDATE

Room 5 is occupied by the admitted patient, so it is marked unavailable. The second statement raises the consultation fee of the general physicians by 100 rupees.

```
UPDATE room SET is_available = 'N' WHERE room_id = 5;

UPDATE doctor
SET consultation_fee = consultation_fee + 100
WHERE dept_id = 5;
```

Output:

```
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

Query OK, 2 rows affected (0.01 sec)
Rows matched: 2  Changed: 2  Warnings: 0
```

DELETE

A new appointment is inserted without giving a status, so the default value is used. It is then removed with DELETE, which brings the table back to its earlier state.

```
INSERT INTO appointment (appointment_id, patient_id, doctor_id, appt_date, reason)
VALUES (16, 9, 108, '2026-11-15', 'Fever review');

SELECT appointment_id, appt_date, status FROM appointment
WHERE appointment_id = 16;

DELETE FROM appointment WHERE appointment_id = 16;
```

Output:

```
Query OK, 1 row affected (0.01 sec)

+----------------+------------+-----------+
| appointment_id | appt_date  | status    |
+----------------+------------+-----------+
|             16 | 2026-11-15 | Scheduled |
+----------------+------------+-----------+
1 row in set (0.00 sec)

Query OK, 1 row affected (0.01 sec)
```

8.3 Data Query Language (DQL)

DQL means retrieving data with SELECT. Twenty detailed queries follow in Chapter 9; the one below shows the basic pattern.

SELECT with column list and WHERE

The most basic form of DQL: choose the columns you need and filter the rows with a condition.

```
SELECT patient_id, first_name, last_name, city, blood_group
FROM patient
WHERE blood_group = 'B+';
```

Output:

```
+------------+------------+-----------+------------+-------------+
| patient_id | first_name | last_name | city       | blood_group |
+------------+------------+-----------+------------+-------------+
|          1 | Amit       | Kapoor    | Chandigarh | B+          |
|          8 | Kavita     | Bansal    | Panchkula  | B+          |
|         11 | Manish     | Tiwari    | Chandigarh | B+          |
+------------+------------+-----------+------------+-------------+
3 rows in set (0.00 sec)
```

8.4 Transaction Control Language (TCL)

A transaction groups statements into one unit of work. COMMIT makes changes permanent, ROLLBACK cancels them and SAVEPOINT marks a point to return to. All tables use the InnoDB engine, which supports transactions.

COMMIT, SAVEPOINT and ROLLBACK TO

A new bill is inserted and a savepoint is created. The pharmacy stock is then reduced by mistake, so the work is rolled back only up to the savepoint. COMMIT finally saves the bill permanently.

```
START TRANSACTION;

INSERT INTO bill VALUES (13, 3, '2026-11-07', 700.00, 'Cash', 'Paid');
SAVEPOINT after_bill;

UPDATE medicine SET stock_qty = stock_qty - 100 WHERE medicine_id = 1;
SELECT medicine_id, medicine_name, stock_qty FROM medicine WHERE medicine_id = 1;

ROLLBACK TO after_bill;
SELECT medicine_id, medicine_name, stock_qty FROM medicine WHERE medicine_id = 1;

COMMIT;
```

Output:

```
Query OK, 0 rows affected (0.01 sec)   -- every non-SELECT statement reports Query OK

-- after the UPDATE, inside the transaction:
+-------------+---------------+-----------+
| medicine_id | medicine_name | stock_qty |
+-------------+---------------+-----------+
|           1 | Paracetamol   |       400 |
+-------------+---------------+-----------+
1 row in set (0.00 sec)

-- after ROLLBACK TO after_bill (stock restored, bill 13 kept):
+-------------+---------------+-----------+
| medicine_id | medicine_name | stock_qty |
+-------------+---------------+-----------+
|           1 | Paracetamol   |       500 |
+-------------+---------------+-----------+
1 row in set (0.00 sec)
```

ROLLBACK

Here all prescription rows are deleted inside a transaction. Because the transaction is rolled back, nothing is lost. The WHERE clause uses the primary key so the statement also works when the Workbench safe-update mode is on.

```
START TRANSACTION;
DELETE FROM prescription WHERE prescription_id > 0;
SELECT COUNT(*) AS rows_inside_transaction FROM prescription;
ROLLBACK;
SELECT COUNT(*) AS rows_after_rollback FROM prescription;
```

Output:

```
Query OK, 0 rows affected (0.01 sec)

Query OK, 18 rows affected (0.01 sec)

+-------------------------+
| rows_inside_transaction |
+-------------------------+
|                       0 |
+-------------------------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)

+---------------------+
| rows_after_rollback |
+---------------------+
|                  18 |
+---------------------+
1 row in set (0.00 sec)
```

8.5 Data Control Language (DCL)

DCL commands (GRANT and REVOKE) decide who may do what. Following the principle of least privilege, each user gets only the rights the job needs. These statements are run from the root account.

CREATE USER, GRANT, REVOKE

Two database users are created: a receptionist who handles registrations and bookings, and a doctor account that can read everything but only write prescriptions. These statements must be run from the root account.

```
CREATE USER 'receptionist'@'localhost' IDENTIFIED BY 'Recep@123';
CREATE USER 'doctor_user'@'localhost' IDENTIFIED BY 'Doctor@123';

GRANT SELECT, INSERT, UPDATE ON medicare_db.patient
    TO 'receptionist'@'localhost';
GRANT SELECT, INSERT ON medicare_db.appointment
    TO 'receptionist'@'localhost';
GRANT SELECT ON medicare_db.* TO 'doctor_user'@'localhost';
GRANT INSERT, UPDATE ON medicare_db.prescription
    TO 'doctor_user'@'localhost';

REVOKE UPDATE ON medicare_db.patient FROM 'receptionist'@'localhost';
SHOW GRANTS FOR 'receptionist'@'localhost';
```

Output:

```
Query OK, 0 rows affected (0.02 sec)   -- CREATE USER (run twice)
Query OK, 0 rows affected (0.01 sec)   -- GRANT (run four times)
Query OK, 0 rows affected (0.01 sec)   -- REVOKE

+-----------------------------------------------------------------------------------+
| Grants for receptionist@localhost                                                 |
+-----------------------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `receptionist`@`localhost`                                  |
| GRANT SELECT, INSERT ON `medicare_db`.`appointment` TO `receptionist`@`localhost` |
| GRANT SELECT, INSERT ON `medicare_db`.`patient` TO `receptionist`@`localhost`     |
+-----------------------------------------------------------------------------------+
3 rows in set (0.00 sec)
```

Chapter 9: SQL Queries with Explanation and Output

Twenty queries on the MediCare database are given below, grouped by topic. For each query the purpose is explained, followed by the code (light blue box) and the output returned by MySQL (grey box).

9.1 Basic retrieval

Query 1: Doctors ordered by salary

Lists every doctor with the full name built using CONCAT and sorts the result from the highest to the lowest salary.

```
SELECT doctor_id,
       CONCAT(first_name, ' ', last_name) AS doctor_name,
       specialization, salary
FROM doctor
ORDER BY salary DESC;
```

```
+-----------+----------------+--------------------+-----------+
| doctor_id | doctor_name    | specialization     | salary    |
+-----------+----------------+--------------------+-----------+
|       101 | Rajesh Kumar   | Cardiologist       | 150000.00 |
|       102 | Anjali Sharma  | Neurologist        | 140000.00 |
|       103 | Vikram Singh   | Orthopedic Surgeon | 135000.00 |
|       107 | Arvind Chauhan | Cardiologist       | 120000.00 |
|       104 | Meera Nair     | Pediatrician       |  95000.00 |
|       106 | Priya Malhotra | Dermatologist      |  90000.00 |
|       105 | Sandeep Verma  | General Physician  |  85000.00 |
|       108 | Neha Gupta     | General Physician  |  80000.00 |
+-----------+----------------+--------------------+-----------+
8 rows in set (0.00 sec)
```

Query 2: Patients from selected cities (IN)

The IN operator keeps the query short when a column has to be matched with several values.

```
SELECT patient_id, first_name, last_name, city
FROM patient
WHERE city IN ('Chandigarh', 'Mohali')
ORDER BY city, patient_id;
```

```
+------------+------------+-----------+------------+
| patient_id | first_name | last_name | city       |
+------------+------------+-----------+------------+
|          1 | Amit       | Kapoor    | Chandigarh |
|          6 | Simran     | Kaur      | Chandigarh |
|         11 | Manish     | Tiwari    | Chandigarh |
|          2 | Sneha      | Reddy     | Mohali     |
|          7 | Deepak     | Joshi     | Mohali     |
|         12 | Ritu       | Saxena    | Mohali     |
+------------+------------+-----------+------------+
6 rows in set (0.00 sec)
```

Query 3: Appointments of October 2026 (BETWEEN)

BETWEEN checks a range of dates, and both end dates are included in the result.

```
SELECT appointment_id, patient_id, doctor_id, appt_date, reason
FROM appointment
WHERE appt_date BETWEEN '2026-10-01' AND '2026-10-31'
ORDER BY appt_date;
```

```
+----------------+------------+-----------+------------+--------------------+
| appointment_id | patient_id | doctor_id | appt_date  | reason             |
+----------------+------------+-----------+------------+--------------------+
|              6 |          4 |       103 | 2026-10-02 | Leg injury         |
|              7 |         11 |       105 | 2026-10-05 | Seasonal flu       |
|              8 |          8 |       102 | 2026-10-12 | Recurring migraine |
|              9 |          2 |       108 | 2026-10-18 | High fever         |
|             10 |          9 |       104 | 2026-10-25 | Vaccination check  |
|             11 |         12 |       106 | 2026-10-30 | Skin rash          |
+----------------+------------+-----------+------------+--------------------+
6 rows in set (0.00 sec)
```

Query 4: Medicines starting with the letter A (LIKE)

The % wildcard stands for any number of characters, so 'A%' finds all medicine names that begin with A.

```
SELECT medicine_id, medicine_name, manufacturer, unit_price
FROM medicine
WHERE medicine_name LIKE 'A%';
```

```
+-------------+---------------+--------------+------------+
| medicine_id | medicine_name | manufacturer | unit_price |
+-------------+---------------+--------------+------------+
|           2 | Amoxicillin   | Sun Pharma   |      12.00 |
|           4 | Atorvastatin  | Lupin        |      18.50 |
|           5 | Aspirin       | Cipla        |       3.00 |
|           8 | Azithromycin  | Zydus        |      25.00 |
|           9 | Amlodipine    | Lupin        |       8.00 |
+-------------+---------------+--------------+------------+
5 rows in set (0.00 sec)
```

Query 5: Bills that are not fully paid

Shows bills whose status is anything other than Paid. The NULL in payment_mode shows that no payment has been made yet.

```
SELECT bill_id, patient_id, bill_date, total_amount,
       payment_mode, payment_status
FROM bill
WHERE payment_status <> 'Paid';
```

```
+---------+------------+------------+--------------+--------------+----------------+
| bill_id | patient_id | bill_date  | total_amount | payment_mode | payment_status |
+---------+------------+------------+--------------+--------------+----------------+
|       9 |          8 | 2026-10-12 |      1100.00 | Card         | Partial        |
|      12 |         10 | 2026-11-01 |      2400.00 | NULL         | Pending        |
+---------+------------+------------+--------------+--------------+----------------+
2 rows in set (0.00 sec)
```

9.2 Aggregate functions

Query 6: Number of patients in each city

GROUP BY collects the rows of each city together and COUNT(*) counts them.

```
SELECT city, COUNT(*) AS total_patients
FROM patient
GROUP BY city
ORDER BY total_patients DESC, city;
```

```
+------------+----------------+
| city       | total_patients |
+------------+----------------+
| Chandigarh |              3 |
| Mohali     |              3 |
| Ambala     |              2 |
| Panchkula  |              2 |
| Rewari     |              2 |
+------------+----------------+
5 rows in set (0.00 sec)
```

Query 7: Salary statistics of all doctors

Five aggregate functions are used together to get a quick summary of the doctor salaries.

```
SELECT COUNT(*) AS total_doctors,
       SUM(salary) AS total_salary,
       ROUND(AVG(salary), 2) AS avg_salary,
       MIN(salary) AS min_salary,
       MAX(salary) AS max_salary
FROM doctor;
```

```
+---------------+--------------+------------+------------+------------+
| total_doctors | total_salary | avg_salary | min_salary | max_salary |
+---------------+--------------+------------+------------+------------+
|             8 |    895000.00 |  111875.00 |   80000.00 |  150000.00 |
+---------------+--------------+------------+------------+------------+
1 row in set (0.00 sec)
```

Query 8: Doctors with two or more appointments (HAVING)

WHERE filters single rows before grouping, whereas HAVING filters the groups after the counting is done.

```
SELECT doctor_id, COUNT(*) AS total_appointments
FROM appointment
GROUP BY doctor_id
HAVING COUNT(*) >= 2
ORDER BY total_appointments DESC, doctor_id;
```

```
+-----------+--------------------+
| doctor_id | total_appointments |
+-----------+--------------------+
|       106 |                  3 |
|       101 |                  2 |
|       102 |                  2 |
|       104 |                  2 |
|       105 |                  2 |
|       107 |                  2 |
+-----------+--------------------+
6 rows in set (0.00 sec)
```

Query 9: Revenue collected by payment mode

Adds up the bill amounts for each payment mode. Bills with no payment mode are left out by the WHERE condition.

```
SELECT payment_mode,
       COUNT(*) AS bills,
       SUM(total_amount) AS total_revenue
FROM bill
WHERE payment_mode IS NOT NULL
GROUP BY payment_mode
ORDER BY total_revenue DESC;
```

```
+--------------+-------+---------------+
| payment_mode | bills | total_revenue |
+--------------+-------+---------------+
| Card         |     2 |      49600.00 |
| Insurance    |     1 |      36200.00 |
| UPI          |     4 |      20800.00 |
| Cash         |     5 |       8850.00 |
+--------------+-------+---------------+
4 rows in set (0.00 sec)
```

9.3 Joins

Query 10: Upcoming appointments (three-table INNER JOIN)

Patient, doctor and appointment tables are joined using the foreign keys to produce a readable list of scheduled visits.

```
SELECT a.appointment_id,
       CONCAT(p.first_name, ' ', p.last_name) AS patient,
       CONCAT('Dr. ', d.last_name) AS doctor,
       a.appt_date, a.reason
FROM appointment a
JOIN patient p ON a.patient_id = p.patient_id
JOIN doctor  d ON a.doctor_id  = d.doctor_id
WHERE a.status = 'Scheduled'
ORDER BY a.appt_date;
```

```
+---------------+-------------+--------------+------------+------------------------+
| appointment_id | patient     | doctor       | appt_date  | reason                 |
+---------------+-------------+--------------+------------+------------------------+
|             14 | Amit Kapoor | Dr. Chauhan  | 2026-11-10 | Follow-up visit        |
|             15 | Simran Kaur | Dr. Malhotra | 2026-11-12 | Skin allergy follow-up |
+---------------+-------------+--------------+------------+------------------------+
2 rows in set (0.00 sec)
```

Query 11: Doctors with their departments

A simple INNER JOIN between doctor and department shows where each doctor works.

```
SELECT d.doctor_id,
       CONCAT(d.first_name, ' ', d.last_name) AS doctor_name,
       d.specialization, dep.dept_name, dep.location
FROM doctor d
INNER JOIN department dep ON d.dept_id = dep.dept_id
ORDER BY dep.dept_name, d.doctor_id;
```

```
+-----------+----------------+--------------------+------------------+----------+
| doctor_id | doctor_name    | specialization     | dept_name        | location |
+-----------+----------------+--------------------+------------------+----------+
|       101 | Rajesh Kumar   | Cardiologist       | Cardiology       | Block A  |
|       107 | Arvind Chauhan | Cardiologist       | Cardiology       | Block A  |
|       106 | Priya Malhotra | Dermatologist      | Dermatology      | Block B  |
|       105 | Sandeep Verma  | General Physician  | General Medicine | Block D  |
|       108 | Neha Gupta     | General Physician  | General Medicine | Block D  |
|       102 | Anjali Sharma  | Neurologist        | Neurology        | Block B  |
|       103 | Vikram Singh   | Orthopedic Surgeon | Orthopedics      | Block C  |
|       104 | Meera Nair     | Pediatrician       | Pediatrics       | Block A  |
+-----------+----------------+--------------------+------------------+----------+
8 rows in set (0.00 sec)
```

Query 12: Patients who were never admitted (LEFT JOIN)

A LEFT JOIN keeps every patient, even those without a matching admission. Rows where admission_id is NULL are the patients who were never admitted.

```
SELECT p.patient_id, p.first_name, p.last_name, p.city
FROM patient p
LEFT JOIN admission adm ON p.patient_id = adm.patient_id
WHERE adm.admission_id IS NULL
ORDER BY p.patient_id;
```

```
+------------+------------+-----------+------------+
| patient_id | first_name | last_name | city       |
+------------+------------+-----------+------------+
|          1 | Amit       | Kapoor    | Chandigarh |
|          3 | Rohan      | Mehta     | Panchkula  |
|          6 | Simran     | Kaur      | Chandigarh |
|          8 | Kavita     | Bansal    | Panchkula  |
|          9 | Harsh      | Vardhan   | Rewari     |
|         11 | Manish     | Tiwari    | Chandigarh |
|         12 | Ritu       | Saxena    | Mohali     |
+------------+------------+-----------+------------+
7 rows in set (0.00 sec)
```

Query 13: Prescriptions written by cardiologists (four-table join)

Starting from prescription, the query travels through appointment, patient, doctor and medicine to show who was given which medicine and by whom.

```
SELECT CONCAT(p.first_name, ' ', p.last_name) AS patient,
       CONCAT('Dr. ', d.last_name) AS doctor,
       m.medicine_name, pr.dosage, pr.duration_days
FROM prescription pr
JOIN appointment a ON pr.appointment_id = a.appointment_id
JOIN patient p     ON a.patient_id = p.patient_id
JOIN doctor d      ON a.doctor_id = d.doctor_id
JOIN medicine m    ON pr.medicine_id = m.medicine_id
WHERE d.specialization = 'Cardiologist'
ORDER BY pr.prescription_id;
```

```
+---------------+-------------+---------------+------------+---------------+
| patient       | doctor      | medicine_name | dosage     | duration_days |
+---------------+-------------+---------------+------------+---------------+
| Amit Kapoor   | Dr. Kumar   | Atorvastatin  | Once daily |            30 |
| Amit Kapoor   | Dr. Kumar   | Aspirin       | Once daily |            30 |
| Karan Malhotra| Dr. Chauhan | Atorvastatin  | Once daily |            30 |
| Karan Malhotra| Dr. Chauhan | Amlodipine    | Once daily |            30 |
| Karan Malhotra| Dr. Chauhan | Aspirin       | Once daily |            30 |
| Ritu Saxena   | Dr. Kumar   | Amlodipine    | Once daily |            30 |
+---------------+-------------+---------------+------------+---------------+
6 rows in set (0.00 sec)
```

Query 14: Room charges of discharged patients

DATEDIFF gives the number of days between the dates. Multiplying it with the daily charge gives the room cost of each stay.

```
SELECT adm.admission_id, p.first_name, r.room_type,
       DATEDIFF(adm.discharge_date, adm.admit_date) AS stay_days,
       DATEDIFF(adm.discharge_date, adm.admit_date)
           * r.charge_per_day AS room_cost
FROM admission adm
JOIN patient p ON adm.patient_id = p.patient_id
JOIN room r    ON adm.room_id = r.room_id
WHERE adm.discharge_date IS NOT NULL
ORDER BY adm.admission_id;
```

```
+--------------+------------+--------------+-----------+-----------+
| admission_id | first_name | room_type    | stay_days | room_cost |
+--------------+------------+--------------+-----------+-----------+
|            1 | Karan      | ICU          |         5 |  40000.00 |
|            2 | Deepak     | Private      |         6 |  24000.00 |
|            3 | Pooja      | Semi-Private |         3 |   7500.00 |
|            4 | Sneha      | General      |         2 |   3000.00 |
+--------------+------------+--------------+-----------+-----------+
4 rows in set (0.00 sec)
```

9.4 Subqueries

Query 15: Doctors earning above the average (scalar subquery)

The inner query calculates the average salary once and the outer query compares every doctor with that value.

```
SELECT doctor_id, first_name, last_name, salary
FROM doctor
WHERE salary > (SELECT AVG(salary) FROM doctor)
ORDER BY salary DESC;
```

```
+-----------+------------+-----------+-----------+
| doctor_id | first_name | last_name | salary    |
+-----------+------------+-----------+-----------+
|       101 | Rajesh     | Kumar     | 150000.00 |
|       102 | Anjali     | Sharma    | 140000.00 |
|       103 | Vikram     | Singh     | 135000.00 |
|       107 | Arvind     | Chauhan   | 120000.00 |
+-----------+------------+-----------+-----------+
4 rows in set (0.00 sec)
```

Query 16: Patients who were prescribed Aspirin (nested subquery)

Three levels of queries are used. The innermost finds the medicine id, the middle one finds the appointments and the outer one finds the patients.

```
SELECT patient_id, first_name, last_name
FROM patient
WHERE patient_id IN (
    SELECT patient_id FROM appointment
    WHERE appointment_id IN (
        SELECT appointment_id FROM prescription
        WHERE medicine_id = (SELECT medicine_id FROM medicine
                             WHERE medicine_name = 'Aspirin')))
ORDER BY patient_id;
```

```
+------------+------------+-----------+
| patient_id | first_name | last_name |
+------------+------------+-----------+
|          1 | Amit       | Kapoor    |
|          5 | Karan      | Malhotra  |
|          7 | Deepak     | Joshi     |
+------------+------------+-----------+
3 rows in set (0.00 sec)
```

Query 17: Patients with an unpaid bill (EXISTS)

EXISTS is a correlated subquery. For every patient the inner query checks if at least one unpaid bill is present.

```
SELECT p.patient_id, p.first_name, p.last_name
FROM patient p
WHERE EXISTS (SELECT 1 FROM bill b
              WHERE b.patient_id = p.patient_id
                AND b.payment_status <> 'Paid')
ORDER BY p.patient_id;
```

```
+------------+------------+-----------+
| patient_id | first_name | last_name |
+------------+------------+-----------+
|          8 | Kavita     | Bansal    |
|         10 | Anita      | Choudhary |
+------------+------------+-----------+
2 rows in set (0.00 sec)
```

9.5 Views and CASE

Query 18: View for pending payments

A view stores a query under a name. The billing counter can now see pending payments with a simple SELECT, without writing the join again.

```
CREATE VIEW v_pending_bills AS
SELECT b.bill_id,
       CONCAT(p.first_name, ' ', p.last_name) AS patient,
       b.total_amount, b.payment_status
FROM bill b
JOIN patient p ON b.patient_id = p.patient_id
WHERE b.payment_status <> 'Paid';

SELECT * FROM v_pending_bills;
```

```
Query OK, 0 rows affected (0.01 sec)

+---------+-----------------+--------------+----------------+
| bill_id | patient         | total_amount | payment_status |
+---------+-----------------+--------------+----------------+
|       9 | Kavita Bansal   |      1100.00 | Partial        |
|      12 | Anita Choudhary |      2400.00 | Pending        |
+---------+-----------------+--------------+----------------+
2 rows in set (0.00 sec)
```

Query 19: Stock level report using CASE

CASE works like an if-else statement inside a query. Medicines with less than 100 units are marked for reordering.

```
SELECT medicine_name, stock_qty,
       CASE
           WHEN stock_qty < 100 THEN 'Reorder Now'
           WHEN stock_qty < 300 THEN 'Low'
           ELSE 'Sufficient'
       END AS stock_status
FROM medicine
ORDER BY stock_qty;
```

```
+---------------+-----------+--------------+
| medicine_name | stock_qty | stock_status |
+---------------+-----------+--------------+
| Azithromycin  |        60 | Reorder Now  |
| Ibuprofen     |        80 | Reorder Now  |
| Amlodipine    |       220 | Low          |
| Atorvastatin  |       250 | Low          |
| Amoxicillin   |       300 | Sufficient   |
| Pantoprazole  |       350 | Sufficient   |
| Cetirizine    |       400 | Sufficient   |
| Paracetamol   |       500 | Sufficient   |
| Aspirin       |       600 | Sufficient   |
+---------------+-----------+--------------+
9 rows in set (0.00 sec)
```

Query 20: Appointments handled by each department

Three tables are joined and grouped to see which department is the busiest. LEFT JOINs keep departments even if they have no appointments.

```
SELECT dep.dept_name, COUNT(a.appointment_id) AS total_appointments
FROM department dep
LEFT JOIN doctor d       ON dep.dept_id = d.dept_id
LEFT JOIN appointment a  ON d.doctor_id = a.doctor_id
GROUP BY dep.dept_name
ORDER BY total_appointments DESC, dep.dept_name;
```

```
+-----------------+--------------------+
| dept_name       | total_appointments |
+-----------------+--------------------+
| Cardiology      |                  4 |
| Dermatology     |                  3 |
| General Medicine |                 3 |
| Neurology       |                  2 |
| Pediatrics      |                  2 |
| Orthopedics     |                  1 |
+-----------------+--------------------+
6 rows in set (0.00 sec)
```

Chapter 10: Constraints, Joins, Subqueries and Aggregate Functions

10.1 Constraints Used

Constraint	Purpose	Example in the project
PRIMARY KEY	Identifies every row uniquely	patient_id in patient
FOREIGN KEY	Links a child row to its parent	doctor.dept_id → department
NOT NULL	A value must be given	patient.last_name
UNIQUE	No duplicate values	department.dept_name
CHECK	Value must satisfy a condition	salary > 0
DEFAULT	Value used when none is given	status = 'Scheduled'
ON DELETE CASCADE	Deletes child rows with parent	prescription → appointment

10.2 Joins, Subqueries and Aggregate Functions

Concept	Description	Used in
INNER JOIN	Returns rows that match in both tables	Queries 10, 11, 13, 14
LEFT JOIN	Keeps all left rows, NULL where no match	Queries 12, 20
Scalar / nested subquery	Query inside a query; one value or a list	Queries 15, 16
Correlated subquery (EXISTS)	Inner query depends on the outer row	Query 17
COUNT, SUM, AVG, MIN, MAX	Summarise many rows into one value	Queries 6, 7, 8, 9, 15, 20
GROUP BY and HAVING	Group rows, then filter the groups	Queries 6, 8, 9, 20

10.3 Testing the Constraints

Each statement below breaks a rule on purpose. MySQL refuses it and returns an error, proving that the data stays protected.

Primary key violation

```
INSERT INTO department VALUES (1, 'Radiology', 'Block E', '0172-4001007');
```

Output:

```
ERROR 1062 (23000): Duplicate entry '1' for key 'department.PRIMARY'
```

Foreign key violation

```
INSERT INTO appointment VALUES (20, 99, 101, '2026-11-20', 'Checkup', 'Scheduled');
```

Output:

```
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint
fails (`medicare_db`.`appointment`, CONSTRAINT `fk_appt_patient` FOREIGN KEY
(`patient_id`) REFERENCES `patient` (`patient_id`))
```

CHECK constraint violation

```
INSERT INTO doctor VALUES
(109, 'Test', 'Doctor', 'Dentist', '9876500109', -5000, 500, '2026-01-01', 1);
```

Output:

```
ERROR 3819 (HY000): Check constraint 'chk_doctor_salary' is violated.
```

Chapter 11: Screenshots of Execution

The screenshots below were taken in MySQL Workbench while running the project and show the statements together with the result grid.

```
480 •  SHOW TABLES;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Tables_in_medicare_db
admission
appointment
bill
department
doctor
medicine
patient
prescription
room

Figure 11.1: SHOW TABLES: the nine tables of medicare_db

```
482 •  DESCRIBE patient;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
patient_id	int	NO	PRI	NULL	
first_name	varchar(30)	NO		NULL	
last_name	varchar(30)	NO		NULL	
gender	char(1)	NO		NULL	
date_of_birth	date	NO		NULL	
phone	varchar(15)	YES	UNI	NULL	
city	varchar(40)	YES	MUL	NULL	
blood_group	varchar(3)	YES		NULL	
email	varchar(60)	YES		NULL	

Figure 11.2: DESCRIBE patient: structure of the patient table

```
484 •  SELECT d.doctor_id,
485            CONCAT(d.first_name, ' ', d.last_name) AS doctor_name,
486            d.specialization, dep.dept_name, dep.location
487    FROM doctor d
488    INNER JOIN department dep ON d.dept_id = dep.dept_id
489    ORDER BY dep.dept_name, d.doctor_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

doctor_id	doctor_name	specialization	dept_name	location
101	Rajesh Kumar	Cardiologist	Cardiology	Block A
107	Arvind Chauhan	Cardiologist	Cardiology	Block A
106	Priya Malhotra	Dermatologist	Dermatology	Block B
105	Sandeep Verma	General Physician	General Medicine	Block D
108	Neha Gupta	General Physician	General Medicine	Block D
102	Anjali Sharma	Neurologist	Neurology	Block B
103	Vikram Singh	Orthopedic Surgeon	Orthopedics	Block C
104	Meera Nair	Pediatrician	Pediatrics	Block A

Figure 11.3: Query 11: doctors with their departments (INNER JOIN)

```
491 •  SELECT a.appointment_id,
492            CONCAT(p.first_name, ' ', p.last_name) AS patient,
493            CONCAT('Dr. ', d.last_name) AS doctor,
494            a.appt_date, a.reason
495    FROM appointment a
496    JOIN patient p ON a.patient_id = p.patient_id
497    JOIN doctor  d ON a.doctor_id  = d.doctor_id
498    WHERE a.status = 'Scheduled'
499    ORDER BY a.appt_date;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

appointment_id	patient	doctor	appt_date	reason
14	Amit Kapoor	Dr. Chauhan	2026-11-10	Follow-up visit
15	Simran Kaur	Dr. Malhotra	2026-11-12	Skin allergy follow-up

Figure 11.4: Query 10: upcoming appointments (three-table join)

```sql
SELECT CONCAT(p.first_name, ' ', p.last_name) AS patient,
       CONCAT('Dr. ', d.last_name) AS doctor,
       m.medicine_name, pr.dosage, pr.duration_days
FROM prescription pr
JOIN appointment a ON pr.appointment_id = a.appointment_id
JOIN patient p     ON a.patient_id = p.patient_id
JOIN doctor d      ON a.doctor_id = d.doctor_id
JOIN medicine m    ON pr.medicine_id = m.medicine_id
WHERE d.specialization = 'Cardiologist'
ORDER BY pr.prescription_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

patient	doctor	medicine_name	dosage	duration_days
Amit Kapoor	Dr. Kumar	Atorvastatin	Once daily	30
Amit Kapoor	Dr. Kumar	Aspirin	Once daily	30
Karan Malhotra	Dr. Chauhan	Atorvastatin	Once daily	30
Karan Malhotra	Dr. Chauhan	Amlodipine	Once daily	30
Karan Malhotra	Dr. Chauhan	Aspirin	Once daily	30
Ritu Saxena	Dr. Kumar	Amlodipine	Once daily	30

Figure 11.5: Query 13: prescriptions written by cardiologists (four-table join)

```sql
SELECT payment_mode,
       COUNT(*) AS bills,
       SUM(total_amount) AS total_revenue
FROM bill
WHERE payment_mode IS NOT NULL
GROUP BY payment_mode
ORDER BY total_revenue DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

payment_mode	bills	total_revenue
Card	2	49600.00
Insurance	1	36200.00
UPI	4	20800.00
Cash	4	8150.00

Figure 11.6: Query 9: revenue by payment mode (GROUP BY)

```sql
SELECT patient_id, first_name, last_name
FROM patient
WHERE patient_id IN (
    SELECT patient_id FROM appointment
    WHERE appointment_id IN (
        SELECT appointment_id FROM prescription
        WHERE medicine_id = (SELECT medicine_id FROM medicine
                             WHERE medicine_name = 'Aspirin')))
ORDER BY patient_id;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

patient_id	first_name	last_name
1	Amit	Kapoor
5	Karan	Malhotra
7	Deepak	Joshi
NULL	NULL	NULL

Figure 11.7: Query 16: patients prescribed Aspirin (nested subquery)

Chapter 12: Conclusion

In this project we designed and built a complete database for a hospital. Starting from the requirements, we identified eight entities and seven relationships, drew the ER diagram, converted it into nine tables and normalised them up to 3NF. The database was implemented in MySQL with DDL, DML, DQL, TCL and DCL commands, and twenty queries showed that it can answer practical questions such as which doctor is busiest, which bills are unpaid and which medicines need reordering.

All nine tables were created and loaded with sample data (6 departments, 8 doctors, 12 patients, 15 appointments, 6 rooms, 5 admissions, 9 medicines, 18 prescription lines and the bills). The twenty queries returned the expected results, the constraint tests were rejected by MySQL as intended, and the transaction demo showed that ROLLBACK restores the data. Such a design can serve small hospitals and clinics for patient, billing and pharmacy records, and as a base for a full application. Its limits are the lack of a front end and the use of sample data only.

The project also taught us things that are hard to learn from theory alone. Constraints save effort because wrong data is stopped at the door. A good schema keeps queries short, while a poor one makes even simple reports painful. Small decisions, such as the order of table creation or using NULL for a patient who has not been discharged, decide whether the database behaves sensibly.

GitHub Repository: https://github.com/ishikapandey1/Medicare-Hospital-Management-System

Chapter 13: Future Scope

- **Front end:** a web or desktop application in Java, Python or PHP so that staff need not write SQL.
- **More modules:** laboratory tests, operation theatre scheduling and ambulance service.
- **Appointment slots:** time-slot management to avoid double booking.
- **Stored procedures and triggers:** automatic stock reduction after a prescription and automatic bill calculation.
- **Insurance and online payment:** claim tracking and payment gateway integration.
- **Security and backup:** encrypted patient data, audit logs and scheduled backups.

Chapter 14: References

[1] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed., Pearson, 2016.

[2] A. Silberschatz, H. F. Korth and S. Sudarshan, Database System Concepts, 7th ed., McGraw-Hill, 2019.

[3] C. J. Date, An Introduction to Database Systems, 8th ed., Pearson, 2003.

[4] R. Ramakrishnan and J. Gehrke, Database Management Systems, 3rd ed., McGraw-Hill, 2003.

[5] Oracle Corporation, MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/

[6] Oracle Corporation, MySQL Workbench Manual. https://dev.mysql.com/doc/workbench/en/

[7] W3Schools, SQL Tutorial. https://www.w3schools.com/sql/

[8] GeeksforGeeks, DBMS Tutorial. https://www.geeksforgeeks.org/dbms/

[9] Graphviz Documentation. https://graphviz.org/documentation/

[10] Chandigarh University, Database Management System course material.